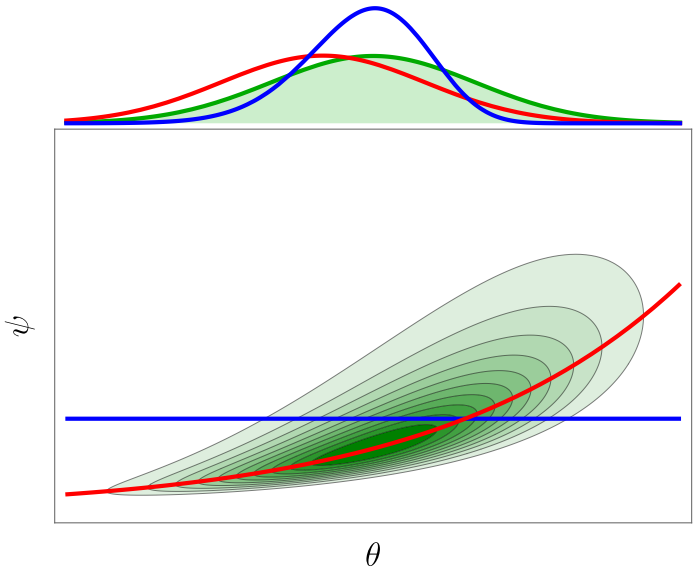

CMP++
Technical Documentation

Version 2.0

Omar Kahol

29 September 2025

Contents

1 Introduction **2**

2 Installation **3**
2.1 Installing Eigen 3
2.2 Installing NLopt 3
2.3 Installing LIBSVM 4
2.4 Installing CMP++ 5

3 Test-cases **6**
3.1 Probability distributions 6
3.2 Markov Chain Monte Carlo 7
3.3 Kernel Density Estimation 9
3.4 Classifier 12
3.5 Gaussian Process Regression 15
3.6 Multi-Output Gaussian Process 16
3.7 Polynomial Chaos Expansion 18
3.8 Optimization 18
3.9 Sobol Sensitivity Analysis 19
3.10 Evolutionary MCMC 20
3.11 Clustering with DPMM 21
3.12 Bayesian Calibration with Model Discrepancy 22

List of Code Listings

1 Introduction

This document is the technical documentation for the library CMP++[1], a library primarily built for performing Bayesian calibration of computer models using several methods:

1. Full Bayes.
2. Adaptive approaches (like CMP and FMP) .
3. Sequential approaches (like KOH).

The library also implements several Uncertainty Quantification (UQ) tools such as Gaussian Processes (GPs), Markov Chain Monte Carlo (MCMC) methods, Polynomial Chaos Expansions (PCE), Sobol sensitivity analysis and much more. The library is written entirely in C++ and currently provides no high level configuration file hence the pre-processing should be done using a C++ application. The post-processing can be performed using any library or software capable of handling .csv input files.

The library can be compiled as a static library or dynamic library and depends on the following libraries:

1. Eigen 3.4.0[2] for linear algebra.
2. NLopt 2.7.1[3] for gradient-free and gradient-based optimization.
3. LIBSVM[4] for support vector machines.
4. Doxygen[5] to generate the documentation.

These libraries should be installed in your system before proceeding. A detailed installation guide is provided in the next section. Furthermore, the test cases provided in the library use Matplotlib++ for plotting the results. Though this library is not required for the compilation of CMP++ it is strongly recommended to install it if you want to run the test cases. If you choose not to install it, you will have to modify the test cases accordingly and save the outputs as CVS files to be post-processed using another software.

[1] https://github.com/omarkahol/cmp.git
[2] https://eigen.tuxfamily.org/index.php?title=Main_Page
[3] https://nlopt.readthedocs.io/en/latest/
[4] https://www.csie.ntu.edu.tw/~cjlin/libsvm/
[5] https://www.doxygen.nl

2 Installation

This section is dedicated to the installation of the CMP++ library and its dependencies. Expert users can skip this section, install the dependencies themselves, compile CMP++ using the provided Makefile and link it to their target application either as a static or dynamic library. The following workflow is, however, strongly recommended.

First and foremost, make sure that you have a C++ compiler and cmake installed in your system. The latter can be checked by calling the following command in the terminal,

```
make --version
```

The output should indicate the version installed. If this is the case, you can proceed to install the dependencies.

2.1 Installing Eigen

Eigen is a header only library so you just need to download the latest version from the official website[6] and copy the folder into a known location. In this guide, I will assume that you have copied it into the folder HOME/opt/eigen-3.4.0 but you can choose whatever location you prefer. To use Eigen in your code, simply include the main header file

```
#include <Eigen/Dense>
```

2.2 Installing NLopt

NLopt is compiled and linked as a dynamic library. This means that we will compile it as .dylib (in MacOs) file and we'll link it to our code during compilation. To install NLopt, you should download the latest version from the official website[7].

[6] https://eigen.tuxfamily.org/index.php?title=Main_Page
[7] https://nlopt.readthedocs.io/en/latest/

To install NLopt in HOME/opt copy the downloaded file in that folder and make sure that the name is something like nlopt-2.7.1 (otherwise simply rename the folder), access it via the terminal and type the following commands.

```
mkdir build
cd build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/opt/nlopt-2.7.1 ..
make
make install
```

This will compile the library into HOME/opt/nlopt-2.7.1 but you can change that by substituting HOME/opt in line 3 with whatever folder you desire. If the installation was successful, you should see that, in the nlopt folder you have an include folder which contains some **header files** and a lib folder which should contain (in MacOs) a **libnlopt.dylib** file. These two folders and files are the only one required so the rest can be safely removed.

Now that we have compiled the library another operation must be performed. It is indeed necessary to append the lib folder path to the system path so that the executable knows where to look for the library during runtime. In MacOs, it is sufficient to add the following line to the .zshrc file in the home folder and restart the terminal

```
export DYLD_LIBRARY_PATH=${DYLD_LIBRARY_PATH}:$HOME/opt/nlopt-2.7.1/lib
```

If this is not properly done, your code will compile and link properly otherwise you will see an *unresolved external symbol* error message at runtime.

2.3 Installing LIBSVM

LIBSVM is another library which is compiled and linked as a dynamic library. To install it, you should download the latest version from the official website[8]. After downloading, compilation and linking is performed in a similar way as for NLopt.

[8] https://www.csie.ntu.edu.tw/~cjlin/libsvm/

2.4 Installing CMP++

In this subsection, I will show how to compile CMP++ as a static and dynamic library. First download the code the official GitHub repository and copy it into a known location. In this guide, I will assume that you have copied it into the folder HOME/opt/CMP++ but you can choose whatever location you prefer. To proceed, open the makefile in the base folder and modify the following lines according to your configuration,

```
# Define the C++ compiler to use
CXX = g++-15

# Define any compile-time flags
CXXFLAGS := -std=c++20 -O3 -fopenmp -mmacosx-version-min=14.0 -DNDEBUG
DEBUG_CXXFLAGS := -std=c++20 -g -O0 -fopenmp -mmacosx-version-min=14.0

# Define external includes
EIGEN = $(HOME)/opt/eigen-3.4.0/
NLOPTINC = $(HOME)/opt/nlopt-2.7.1/include
SELF = ./include
USR = /opt/homebrew/include
OMPINC = $(HOME)/opt/openmp/include
SVMINC = $(HOME)/opt/libsvm

# Define external libs
NLOPTLIB = $(HOME)/opt/nlopt-2.7.1/lib
OMPLIB = $(HOME)/opt/openmp/lib
SVMLIB = $(HOME)/opt/libsvm
```

The first variable should be modified to reflect the C++ compiler installed in your system. The second one contains the compilation flags which can be modified according to your needs. Right now CMP can be compiled both with debug flags and without, that's why you see two different CXXFLAGS variables. The next lines contain the path of the header and lib folders of the dependencies we installed in the previous sections. Make sure to modify them according to your configuration.

Finally, to compile CMP++ as a static library, simply access the CMP++ folder via the terminal and type *make release* to compile it without debug flags,

make debug or *make* to compile both versions.

If the compilation was successful, the lib folder should contain two files **libcmp.a** and **libcmp.dylib** which are the static and dynamic library respectively. If the debug version was compiled, the lib folder should also contain **libcmp_debug.a** and **libcmp_debug.dylib**.

This step should also generate the Doxygen documentation for the code in a folder called docs. To link against CMP++ in your code you should include the header files and link against the library. This can be done by adding the following lines to a makefile

```
# Define CMP++ includes
CMPINC = $(HOME)/opt/CMP++/include
CMPLIB = $(HOME)/opt/CMP++/lib
# Linking against CMP++ static library
LDFLAGS = -lcmp
```

Make sure to replace HOME/opt/CMP++ with the path where you have copied CMP++. See the provided test cases for more details.

3 Test-cases

3.1 Probability distributions

This test-case is designed to showcase the distribution module of CMP++. In particular, we will show how to create a multivariate normal distribution, sample from it and compute the lagged correlation of the samples. The test case is contained in the file **distribution.cpp**. Right now CMP++ provides many probability distributions such as univariate and multivariate normal, uniform, exponential, gamma, inverse gamma, beta and many more. The complete list can be found in the documentation[9]. These distributions implement many useful functions such as sampling, pdf, cdf, icdf and also the gradient and hessian of the log-pdf. They can be used to create priors for the parameters and hyperparameters or to create proposal distributions for MCMC simulations.

[9] https://omarkahol.github.io/cmp/reference/html/

First let's create a 2D multivariate normal distribution,

```
// Define a 2D mean and covariance matrix
     // Define a 2D mean and covariance matrix
    Eigen::VectorXd mean(2);
    mean << 0.0, 0.0;
    Eigen::MatrixXd cov(2, 2);
    cov << 1.0, 0.8,
        0.8, 1.0;

    // Create the Multivariate Normal Distribution
    cmp::distribution::MultivariateNormalDistribution mvn(mean,
     ↪  cov);
```

This creates a multivariate normal distribution. The object can be used to extract some samples from the distribution. Using Matplotlib we can plot the samples. Additionally, in *statistics.h* there is a function to compute the lagged correlation of the samples.

Try experimenting with different distributions and different parameters. Also, what happens if you increase the number of samples? How does the lagged correlation change?

3.2 Markov Chain Monte Carlo

This test-case is designed to showcase the MCMC module of CMP++. In particular, we will show how to sample a complex posterior distribution using the Metropolis algorithm with Delayed Rejection and Adaptive Metropolis (DRAM) and a multivariate t-distribution as proposal. The test case is contained in the file **mcmc.cpp**. The MCMC module implements many useful functions for diagnostics as well as a DiffeRential Evolution Adaptive Metropolis (DREAM) class. The target distribution is a bimodal distribution made from the following function

$$\log\ \mathrm{p}(\mathrm{x}) \propto -\gamma\left(\mathrm{x}^2-1\right)^2\ . \tag{1}$$

Increasing the value of gamma will make the two modes more pronounced.

First let's create a proposal distribution and build our MCMC object

```

// Create the distribution object
cmp::distribution::MultivariateStudentDistribution proposal(xInit, covInit, 1);

// Create the MCMC chain object
cmp::mcmc::MarkovChain chain(&proposal, rng);

```

The chosen distribution is a multivariate t-distribution with 1 degree of freedom which is equivalent to a Cauchy distribution. This distribution has heavier tails than a normal distribution and is therefore more suitable for sampling multimodal distributions. The MarkovChain object is then created by passing the proposal and a random number generator.

To sample we just need to call the *step* method by passing the target function and a few additional parameters for the DRAM step (or set the bool input to false to use a simple Metropolis algorithm). We choose to discard the first 10,000 samples as burn-in and then draw 100,000 samples. To reduce the correlation between the samples we choose to subsample every 10 samples.

We then plot the samples and observe the two modes captured correctly by the MCMC and compute the lagged correlation of the samples and the self correlation length and effective sample size using the functions implemented in statistics.h.

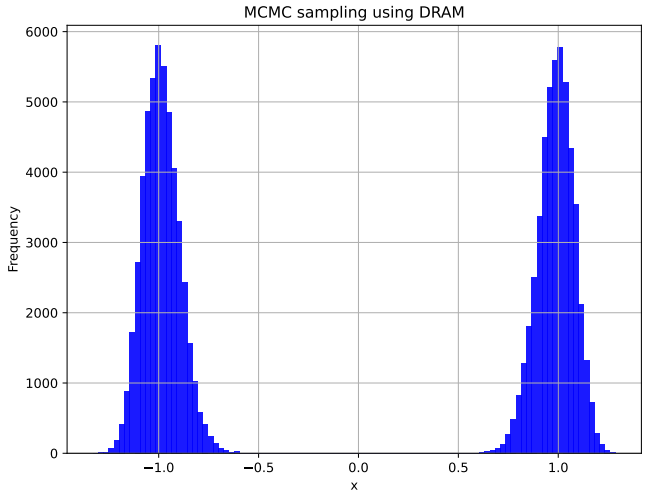

Figure 1: Samples from the bimodal distribution using MCMC with DRAM and a multivariate t-distribution as proposal.

Try the following experiments

1. Use a Multivariate Normal distribution with a smaller covariance matrix. Can you still capture the two modes?
2. Remove the DRAM step. How does the acceptance rate change?
3. Increase or decrease the subsampling factor. How does the self correlation length and effective sample size change?

3.3 Kernel Density Estimation

Kernel Density Estimation (KDE) is a non-parametric way to estimate the probability density function of a random variable. It is a fundamental data smoothing problem where inferences about the population are made based on a finite data sample. The test case is contained in the file **kde.cpp**. In this test-case we will try to estimate the density of a Mixture of two Gaussians (MoG) using KDE.

First we construct a Mixture of two Gaussians using the MultivariateMixture-Distribution class,

```
// Create the mixture distribution
cmp::distribution::MultivariateMixtureDistribution mixture({
    std::make_shared<cmp::distribution::MultivariateNormalDistribution>(mean
    cov1),
    std::make_shared<cmp::distribution::MultivariateNormalDistribution>(mean
    cov2)
}, {0.3, 0.7}); // weights
```

In the code snippet above we create a mixture of two Gaussians with weights 0.3 and 0.7. We then draw 1000 samples from the distribution using the sample method and plot the results using Matplotlib++.

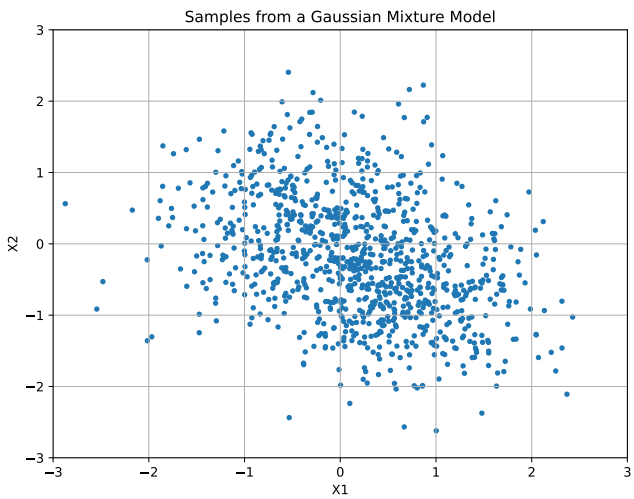

Figure 2: Samples from the Mixture of two Gaussians.

Fig. 2 shows the samples drawn from the MoG. Now we can proceed to fit a KDE to the samples. To define a KDE we need to select an appropriate kernel and a bandwidth. The philosophy behind CMP++ is to keep everything as modular as possible, so the user is free to choose the most appropriate model. In CMP++ the bandwidth is a class which inherits from a base class

called Bandwidth. Multiple bandwidth models are already implemented such as FullBandwidth, DiagonalBandwidth and ScalarBandwidth. User is also free to implement their own bandwidth model, for example an anisotropic bandwidth, function of the location. Similarly, the kernel is also a class which inherits from a base class called Kernel. Multiple kernels are already implemented such as Gaussian, Epanechnikov, Uniform and many more. User is also free to implement their own kernel.

In this test case we choose to use a Gaussian kernel and a full bandwidth. First we estimate the bandwidth matrix purely from the samples using Silverman's rule and initialize a corresponding FullBandwidth and Gaussian object

```
/// First we estimate the bandwidth using Silverman's rule of thumb
Eigen::Matrix2d sample_band = cmp::density::bandwidthSelectionRule(cmp::density::method::SILVERMAN, samples);

// Create the bandwidth and kernel objects
auto bandwidth = std::make_shared<cmp::kernel::FullBandwidth>(sample_band);
auto kernel = cmp::kernel::Gaussian::make();

// Set up the KDE
cmp::density::KDE kde;
kde.set(bandwidth, kernel, samples);
```

We can now evaluate the density at any point in the domain. For reference we also compare it to the true density. Now we see that, although close, the estimated density is not very accurate. This is because the bandwidth was estimated purely from the samples and not optimized to minimize some error metric. A popular approach to improve the bandwidth is to maximize the log-likelihood of the samples using cross-validation. This is implemented in CMP++ in the function *bandwidthOptimizationCrossValidation* which takes as input a KFold object, the samples, the kernel, the bandwidth to optimize, the lower and upper bound for the optimization, the nlopt algorithm to use and a tolerance. The KFold object is used to split the samples into training and validation sets. The function will then optimize the bandwidth by maximizing the log-likelihood of the validation set given the training set, repeating

the process for each fold.

```
// Now we improve the bandwidth using likelihood cross-validation
cmp::statistics::KFold kf(20, nSamples, true, 42);
cmp::density::bandwidthOptimizationCrossValidation(kf, samples, kernel, bandwidth, -1, 1, nlopt::LD_TNEWTON, 1e-3);
```

In the above code we have created a KFold object with 20 folds and then called the optimization function. Since the bandwidth is treated as a pointer, the optimized bandwidth is automatically updated. We can now re-evaluate the density and compare it to the true density.

You can try the following experiments

1. Change the kernel to Epanechnikov or Uniform. How does the estimated density change?
2. Change the bandwidth to Diagonal or Scalar. How does the estimated density change?
3. Change the number of folds in the KFold object. How does the estimated density change and the optimization time?

3.4 Classifier

This test-case is designed to showcase the different classifiers implemented in CMP++. Currently we offer

1. Support Vector Machines (SVM).
2. k-Nearest Neighbors (kNN).
3. Naive Bayes.
4. Kernel Density Estimation (KDE).

The test case is contained in the file **classifier.cpp**. In this test-case we will implement a simple classification task using the different classifiers. We will generate a synthetic dataset using the well known 2D moons dataset. The

dataset is generated using the function *make_moons* which takes as input the number of samples, the noise level and a random seed.

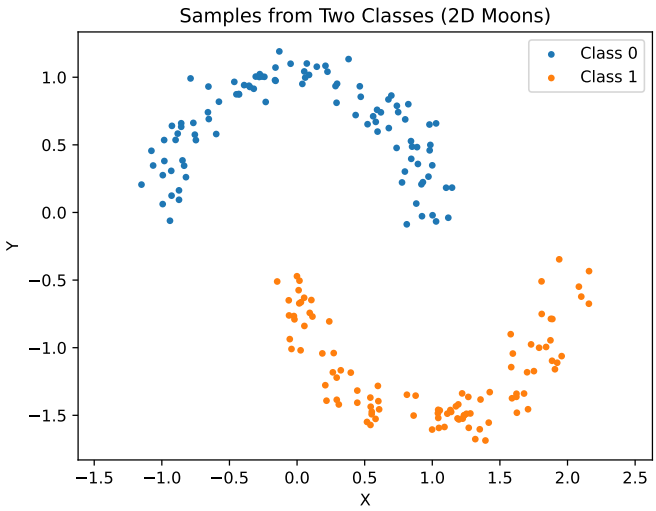

Figure 3: 2D moons dataset with 200 samples and noise level 0.1.

We then create the different classifiers and fit them to the training set. The KDE classifier uses a Gaussian kernel and a full bandwidth which is selected using Silverman's rule of thumb. The naive Bayes classifier uses a Gaussian distribution for each class. The kNN classifier uses 5 neighbors and the SVM classifier uses a radial basis function (RBF) kernel with default parameters.

```cpp
// Make a classifier --- KDE ---
cmp::classifier::KDE kde;
cmp::kernel::Gaussian kernel;

// Estimate the bandwidth using Silverman's rule of thumb
Eigen::Matrix2d sample_band = cmp::density::bandwidthSelectionRule(cmp::density::method::SILVERMAN, samples);
cmp::kernel::FullBandwidth bandwidth(sample_band);

// Set the KDE classifier
kde.set(std::make_shared<cmp::kernel::Gaussian>(kernel), std::make_shared<cmp::kernel::FullBandwidth>(bandwidth), samples, labels);

// Make a classifier --- Naive Bayes ---
cmp::classifier::NaiveBayes nb;
nb.fit(samples, labels);

// Make a classifier --- SVM ---
cmp::classifier::SVM svm(1, 100);
svm.fit(samples, labels);
```

The decision boundary, with a 50% confidence interval, is then plotted for each classifier.

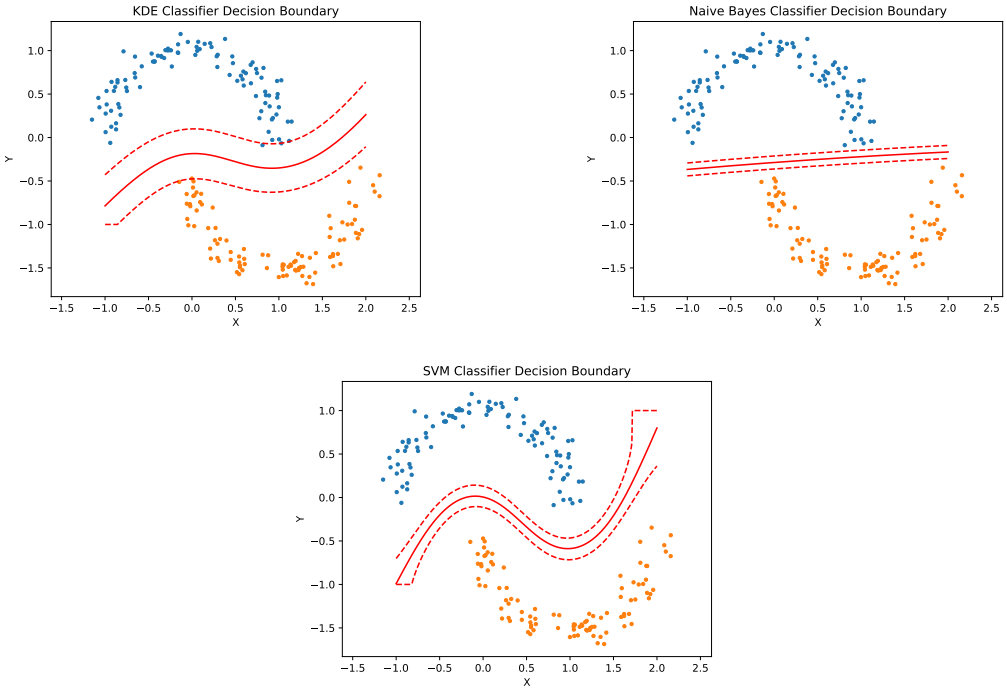

Figure 4: Decision boundaries for different classifiers on the 2D moons dataset.

As we can see, the SVM classifier performs the best on this dataset.

3.5 Gaussian Process Regression

This test-case is designed to showcase the Gaussian Process module of CMP++. The test case is contained in the file **gp.cpp**. In this example we generate synthetic observations from

$$\mathrm{y}(\mathrm{x}) = \sin(2\pi\mathrm{x}) + 1 + \varepsilon,$$

where ε is small Gaussian noise, and we model the data with a GP using a Constant $\times$ Squared Exponential kernel and a constant mean.

The workflow shown in this test is: (1) define kernel and mean, (2) draw prior samples, (3) condition on observations to obtain posterior samples, (4) evaluate predictive quality with K-fold cross-validation, and finally (5) optimize hyperparameters using NLopt.

```
auto kernel = cmp::covariance::Constant::make(0) * cmp::covariance::SquaredExponential::make(1, -1);
auto mean = cmp::mean::Constant::make(2);

cmp::gp::GaussianProcess gp;
gp.set(kernel, mean, 0.1 * Eigen::VectorXd::Ones(3), 1e-5);

gp.condition(x_obs, y_obs, false);
auto [mu_post, cov_post] = gp.predictMultiple(x_test);

gp.fit(x_obs, y_obs, lb, ub, cmp::gp::LOO, nlopt::LD_TNEWTON, 1e-5, true, prior);
```

Try the following experiments

1. Replace the constant mean with a zero mean and compare posterior behavior near boundaries.
2. Increase the observation noise and check how the posterior uncertainty changes.
3. Compare MSE before and after hyperparameter optimization for different fold numbers.

3.6 Multi-Output Gaussian Process

This test-case is designed to showcase multi-output GP regression. The test case is contained in **multi_gp.cpp**. In this example we generate three outputs from a shared 1D input, where the third output is constructed from the first two. This setup highlights the benefit of dimensionality reduction before GP fitting.

The code first scales the input with a standard scaler, then applies PCA to the output, and finally fits one GP per retained output component through the `MultiOutputGaussianProcess` interface.

```
cmp::scaler::StandardScaler x_scaler;
auto x_obs_scaled = x_scaler.fit_transform(x_obs);

cmp::scaler::PCA y_scaler(2);
auto y_obs_scaled = y_scaler.fit_transform(y_obs);

cmp::gp::MultiOutputGaussianProcess gp(y_obs_scaled.cols(), kernel, mean,
                                                                    0.1 * Ei
```

Try the following experiments

1. Change the number of PCA components and evaluate prediction quality.
2. Remove PCA and fit directly in output space; compare runtime and stability.
3. Increase output noise and observe the estimated hyperparameters of each GP.

3.7 Polynomial Chaos Expansion

This test-case is designed to showcase the Polynomial Chaos Expansion (PCE) module. The test case is contained in **poly.cpp**. A 1D Legendre-based expansion is fitted to samples generated from a sinusoidal function, and predictions are plotted together with confidence intervals computed from predictive variance.

```
cmp::PolynomialExpansion pce1;
pce1.set(1, 3, cmp::LegendrePolynomial::make());
pce1.fit(data.leftCols(1), data.col(1));

auto [mean, var] = pce1.predict(testGrid.row(i));
```

Try the following experiments

1. Increase polynomial degree and study over/under-fitting behavior.
2. Replace Legendre with Hermite polynomials for Gaussian input assumptions.
3. Add noise to training data and inspect confidence interval width.

3.8 Optimization

This test-case is designed to showcase the optimization wrappers around NLopt. The test case is contained in **optimization.cpp**. Two constrained problems are solved: a Rosenbrock-type problem with inequality constraints and an exponential objective with an equality constraint.

The central component is the `ObjectiveFunctor`, where objective and constraints are defined once and passed to `cmp::nlopt_max` with bounds, algorithm, and tolerance.

```
cmp::ObjectiveFunctor functor(objectiveFunction);
functor.addInequalityConstraint(constraint);
cmp::nlopt_max(functor, x_opt, lb, ub, nlopt::LD_MMA, 1e-6);

cmp::ObjectiveFunctor functor2(objective_grad);
functor2.addEqualityConstraint(constraint_grad);
cmp::nlopt_max(functor2, x_opt2, lb2, ub2, nlopt::LD_SLSQP,
↪  1e-6);
```

Try the following experiments

1. Compare multiple NLopt algorithms for the same constrained problem.
2. Tighten tolerance and monitor objective and gradient norms.
3. Change initial guesses to analyze sensitivity and local convergence.

3.9 Sobol Sensitivity Analysis

This test-case is designed to showcase Sobol global sensitivity analysis with Saltelli sampling. The test case is contained in **sobol.cpp**. The Ishigami function is used as benchmark because first-order, second-order and total-order Sobol indices are known analytically.

The workflow is: generate Saltelli evaluation grid, evaluate the model, compute Sobol indices, and optionally estimate uncertainty through bootstrap.

```
cmp::sobol::SobolSaltelli ss;
auto evalGrid = ss.evaluationGrid(n_points,
        std::make_shared<cmp::grid::ScrambledSobolGrid>(lowerBound, upperBound, 42), true);

auto [meanResults, varResults] = ss.computeWithBootstrap(Y, nBootstrap, 42);
```

Try the following experiments

1. Increase the number of points and compare convergence to analytical indices.
2. Disable second-order terms and compare computational cost.
3. Increase bootstrap samples for tighter confidence intervals.

3.10 Evolutionary MCMC

This test-case is designed to showcase the Evolutionary MCMC implementation. The test case is contained in **emcmc.cpp**. Multiple chains evolve simultaneously and propose jumps using differential-evolution-style moves, improving exploration in multimodal targets.

```
cmp::mcmc::EvolutionaryMarkovChain emchain(initialSamples, initialScores, 1e-2);

for(size_t i = 0; i < n_samples; i++) {
        double gamma = (i % 10 == 0) ? 1.0 : 2.38 / std::sqrt(2.0);
        emchain.step(score, rng, gamma);
}
```

Try the following experiments

1. Increase the number of chains and inspect mixing improvements.

2. Modify the jump scaling schedule and compare mode-switching frequency.
3. Compare EMCMC against standard single-chain DRAM on the same target.

3.11 Clustering with DPMM

This test-case is designed to showcase clustering with a Dirichlet Process Mixture Model (DPMM). The test case is contained in **cluster.cpp**. A two-moons dataset is generated, initialized with geometric clustering, and then refined through Gibbs updates in the DPMM.

```
cmp::cluster::GeometricCluster kmeans;
kmeans.fit(X, n_init_clusters, rng);

cmp::distribution::NormalInverseWishartDistribution hyper(mu0, kappa0, nu0, psi0);
cmp::cluster::DirichletProcessMixtureModel dpmm(alpha, hyper, 42);
dpmm.condition(X, kmeans.getLabels());

for(size_t iter = 0; iter < n_iters; iter++) {
        dpmm.step();
}
```

Try the following experiments

1. Change concentration parameter α and observe the number of inferred clusters.
2. Compare random initialization against geometric clustering initialization.
3. Increase Gibbs iterations and check label stability.

3.12 Bayesian Calibration with Model Discrepancy

This test-case is designed to showcase a compact Bayesian calibration workflow. The test case is contained in **calibration.cpp**. Synthetic data are generated from a parabola and a simple model $\mathrm{y} = \mathrm{c}\mathrm{x}^2$ is calibrated while a GP captures residual model discrepancy.

The unknowns are the model scale c and GP hyperparameters $(\sigma_\mathrm{f}, \ell)$ sampled with MCMC. The test then visualizes posterior histograms and compares synthetic data with the calibrated mean model.

```
auto model = [&](const double& x, const double& model_scale) {
        return model_scale * x * x;
};

cmp::gp::GaussianProcess model_error;
model_error.set(kernel, mean, gp_hyper_guess, observation_noise);
model_error.condition(x_data, compute_residuals(model_scale_guess), true);

cmp::mcmc::MarkovChain mcmc(&proposal, rng, log_posterior(params));
```

Try the following experiments

1. Increase synthetic noise and inspect posterior spread for c.
2. Change prior variance for c and evaluate prior sensitivity.
3. Replace the quadratic model with a linear model and compare model discrepancy behavior.